Generic checklist for building Android games

Flutter + Flame as the primary stack · basis for the "Android game creation" skill

13 phases · 94 items | Priorities: critical · important · nice-to-have | agnostic / flutter-flame | ⚠ verify still current

Prepared for Juan Manuel de Villeros

October 19, 2026 (Bogotá time, UTC-5)

Distilled from three in-house projects and the official Android, Google Play, Flutter and Flame guidelines

How to use this checklist

- Each item has a checkbox, an imperative action, **Why**, **Verify** (command, file or Play Console location), **Priority** and an official **Source** where one exists.
- **Priority:** `critical` blocks publishing or breaks the experience on current devices; `important` is required or strongly recommended by the quality guidelines; `nice-to-have` is an improvement.
- **Scope:** `agnostic` applies to any stack (Unity, Godot, native Kotlin...); `flutter/flame` is specific to the primary stack.
- ⚠ **Verify still current** marks dated requirements that change year to year (target API, 16 KB, policies). Check them against the source before every launch.
- File names follow the `flutter create` template (`android/app/...`, `lib/...`).

Summary by phase (primary priority of each item)

Phase	Items	Critical	Important	Nice-to-have
Phase 0 · Kickoff, template and methodology	9	1	6	2
Phase 1 · Project setup and build	10	5	4	1
Phase 2 · Screen, scaling, orientation, edge-to-edge and large screens	9	2	6	1
Phase 3 · Lifecycle, state and saving	7	1	4	2
Phase 4 · Audio	6	1	3	2
Phase 5 · Input and predictive back	6	0	4	2
Phase 6 · Performance	6	0	4	2
Phase 7 · Accessibility and internationalization	6	0	3	3
Phase 8 · Errors, crashes and symbols	5	0	5	0
Phase 9 · Testing and CI	8	0	6	2
Phase 10 · Signing and release	6	5	0	1
Phase 11 · Google Play publishing and policies	10	6	3	1
Phase 12 · Post-launch maintenance	6	1	3	2
Total	**94**	**22**	**51**	**21**

Phase 0 Kickoff, template and methodology

- [ ] **Write the intent and the spec before the code** (intent → spec → plan → decisions). agnostic
  - Why: an agent or developer without a spec produces rework; the spec is the "done" contract.
  - Verify: `docs/00_intent.md`, `docs/00_spec.md`, `docs/01_plan.md` and `docs/02_decisions.md` (ADRs) exist.
  - Priority: important.

- [ ] **Keep a resumable state file** (current phase, last green checkpoint, blockers, next step). agnostic
  - Why: lets human or agent sessions resume without losing context.
  - Verify: `docs/03_state.md` updated at the close of each phase.
  - Priority: important.

- [ ] **Define agent rules and stop rules**: Play, payments, secrets or keystore, merges to `main` and public releases are always the owner's call. agnostic
  - Why: prevents irreversible actions taken by automation.
  - Verify: `AGENTS.md` with prohibitions, autonomy level and budgets.
  - Priority: important.

- [ ] **Create a single verification harness** (`verify.sh`) used the same way by the agent and by CI. agnostic
  - Why: one definition of "green" keeps any check from being left out.
  - Verify: `./verify.sh` exits 0; `./verify.sh --no-apk` for fast iteration.
  - Priority: important.

- [ ] **Write a human acceptance script** containing only what a machine can't judge (feel, difficulty curve, legibility). agnostic
  - Why: separates automated validation from taste and makes the demo repeatable.
  - Verify: `docs/04_acceptance.md` with checkboxes and a verdict.
  - Priority: nice-to-have.

- [ ] **Lock in the final `applicationId` before the first upload to Play** (your own reverse domain). agnostic
  - Why: it can't be changed once uploaded.
  - Verify: `applicationId` and `namespace` in `android/app/build.gradle.kts`, plus the package of `MainActivity.kt`.
  - Priority: critical.
  - Source: https://docs.flutter.dev/deployment/android

- [ ] **Check that the game's name is available** (Play, domain, trademarks). agnostic
  - Why: renaming later costs the listing, icon, domain and policy.
  - Verify: search Play and the web; domain for the privacy policy.
  - Priority: nice-to-have.

- [ ] **Centralize game constants and strings** (`game_config.dart`, `app_strings.dart`). flutter/flame
  - Why: makes balance tuning, tests and later translation easier.
  - Verify: `rg "fontSize:|Duration\(" lib` shows no scattered magic numbers in the logic.
  - Priority: important.

- [ ] **Separate pure logic (no Flame or Flutter) from components.** flutter/flame
  - Why: pure logic is tested with fast tests that need no platform.
  - Verify: `lib/logic` or `lib/managers` don't import `package:flame` and have tests.
  - Priority: **important**.

Phase 1 Project setup and build

- [ ] **Use Flutter's default SDK values, or pin them deliberately** (`compileSdk`, `targetSdk`, `minSdk`). flutter/flame ⚠ verify still current
  - Why: `flutter.targetSdkVersion` follows the Play requirement as Flutter is updated. A pinned number must be reviewed every year.
  - Verify: `android/app/build.gradle.kts`; effective value in `build/app/intermediates/merged_manifests/release/.../AndroidManifest.xml` (`<uses-sdk>`).
  - Priority: critical.
  - Source: https://docs.flutter.dev/deployment/android

- [ ] **Target the API level Play requires**: since **Aug 31, 2026, API 36** for new apps and updates; existing apps need **API 35 or higher** to stay visible. agnostic ⚠ verify still current
  - Why: Play rejects the upload if it isn't met.
  - Verify: merged manifest (`targetSdkVersion="36"`); Play Console › App bundle explorer.
  - Priority: critical.
  - Source: https://support.google.com/googleplay/android-developer/answer/11926878 · https://developer.android.com/google/play/requirements/target-sdk

- [ ] **Use a 16 KB-compatible toolchain** (AGP 8.5.1 or newer, NDK r28 or newer, Flutter 3.35 or newer) and update plugins that ship native code. agnostic ⚠ verify still current
  - Why: Play requires 16 KB support for apps targeting 35+; non-compliant updates are blocked starting Feb 1, 2027.
  - Verify: `unzip` the AAB and run `readelf -lW base/lib/arm64-v8a/*.so | grep LOAD` (alignment 0x4000 or higher); Play Console › App bundle explorer.
  - Priority: critical.
  - Source: https://developer.android.com/guide/practices/page-sizes

- [ ] **Version from `pubspec.yaml`** (`version: X.Y.Z+N`), bump `N` (versionCode) for every AAB, and keep a single version source in the docs. flutter/flame
  - Why: Play rejects a repeated versionCode, and contradictory versions in the docs cause confusion.
  - Verify: `grep ^version pubspec.yaml`; the README and plan don't cite another version.
  - Priority: critical.

- [ ] **Rely on R8 (on by default in release) and add `proguard-rules.pro` only if a plugin asks for it.** flutter/flame
  - Why: shrinks the build; unnecessary rules hide problems.
  - Verify: `flutter build appbundle` with no R8 errors; the app launches in release.
  - Priority: important.
  - Source: https://docs.flutter.dev/deployment/android

- [ ] **Declare only the permissions the code actually uses**; don't add `INTERNET` if the game is offline. agnostic
  - Why: Minimize_Permissions in the quality guidelines; the privacy policy and Data safety must also match.
  - Verify: `grep uses-permission` in the merged manifest; search `lib/` for real use of each permission (for example, `WAKE_LOCK` without a wakelock plugin is unneeded).
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Remove unused dependencies and bundle fonts instead of downloading them** (with their license, e.g. OFL). flutter/flame
  - Why: a font downloaded at runtime needs network access and changes the privacy surface.
  - Verify: `rg "import 'package:<dep>" lib` for each dependency; `flutter pub deps`; fonts in `assets/fonts/`.
  - Priority: nice-to-have.

- [ ] **Declare every asset in `pubspec.yaml`** and check that it makes it into the build. flutter/flame
  - Why: an undeclared asset only fails at runtime.
  - Verify: `unzip -l build/app/outputs/flutter-apk/app-release.apk | grep flutter_assets`.
  - Priority: important.

- [ ] **Create your own adaptive icon**: legacy, `mipmap-anydpi-v26/ic_launcher.xml` with foreground and background, and a `<monochrome>` layer. agnostic
  - Why: the template's default icon isn't publishable, and Android 13+ uses the monochrome layer for themed icons.
  - Verify: `md5sum` of `mipmap-*/ic_launcher.png` differs from the Flutter template's; the adaptive XML exists.
  - Priority: critical (own icon) / nice-to-have (monochrome).
  - Source: https://docs.flutter.dev/deployment/android

- [ ] **Give `LaunchTheme` and `NormalTheme` the game's background color** (not `Theme.Light` if the game is dark). flutter/flame
  - Why: avoids the white flash before the first frame.
  - Verify: `values/styles.xml`, `values-night/styles.xml` and `drawable*/launch_background.xml` use `@color/splash_background`; cold start in light mode.
  - Priority: important.

```
<!-- values/colors.xml -->
<color name="splash_background">#FF0A0A14</color>
<!-- drawable/launch_background.xml -->
<layer-list xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:drawable="@color/splash_background" />
</layer-list>
```

Phase 2 Screen, scaling, orientation, edge-to-edge and large screens

- [ ] **Declare the app as a game**: `android:appCategory="game"` on `<application>`. agnostic ⚠ verify still current
  - Why: when targeting 36, Android 16 ignores `screenOrientation`, `resizeableActivity` and aspect-ratio limits on screens of 600 dp or more, **except for games**.
  - Verify: merged manifest; a tablet or foldable emulator on API 36 honors the orientation.
  - Priority: critical.
  - Source: https://developer.android.com/about/versions/16/behavior-changes-16 · https://developer.android.com/develop/adaptive-apps/guides/app-orientation-aspect-ratio-resizability

```
<application
    android:label="MyGame"
    android:appCategory="game"
    android:enableOnBackInvokedCallback="true"
    ...>
```

- [ ] **Lock orientation in two places**: the manifest (`portrait` or `sensorLandscape`) and `SystemChrome.setPreferredOrientations`. flutter/flame
  - Why: the manifest prevents the initial rotation and Dart enforces it at runtime.
  - Verify: rotate the device on every screen.
  - Priority: important.

- [ ] **Design for edge-to-edge**: it's mandatory when targeting 35/36, and in Flutter, when targeting 36, modes other than `edgeToEdge` have no effect. Use `SystemUiMode.edgeToEdge` with transparent bars and don't use `windowOptOutEdgeToEdgeEnforcement`. flutter/flame ⚠ verify still current
  - Why: if you assume `immersiveSticky`, the HUD can end up under the bars or the notch.
  - Verify: `rg "setEnabledSystemUIMode|windowOptOutEdgeToEdge" lib android`; test on a physical Android 16 device with both gesture and button navigation.
  - Priority: critical.
  - Source: https://api.flutter.dev/flutter/services/SystemChrome/setEnabledSystemUIMode.html · https://docs.flutter.dev/release/breaking-changes/default-systemuimode-edge-to-edge

```
SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
SystemChrome.setSystemUIOverlayStyle(const SystemUiOverlayStyle(
  statusBarColor: Colors.transparent,
  systemNavigationBarColor: Colors.transparent,
));
```

- [ ] **Respect insets** (bars, notch, cutouts, gesture areas) in the HUD, menus and touch controls: `SafeArea` or `MediaQuery.paddingOf`, with symmetric horizontal margins. flutter/flame
  - Why: a joystick or button under the notch or the gesture area stops working.
  - Verify: emulator with a cutout (Settings › Developer options › Display cutout); widget test with simulated `padding`.
  - Priority: important.

- [ ] **Separate the playfield from the HUD by construction**: no touch-absorbing panel sits on top of the play area. agnostic
  - Why: playable elements under the HUD can't be touched; it's a typical failure of full-bleed mockups.
  - Verify: widget test at 3 or 4 sizes (e.g. 360×640, 412×915, 800×360, tablet): playfield ∩ HUD = ∅ and the hit test reaches the game's `Listener`.
  - Priority: important.

- [ ] **Scale the game world proportionally** (camera or `FixedResolutionViewport`, `FittedBox` for fixed boards, scale by the short side) and test several aspect ratios (16:9 to 21:9, tablet). flutter/flame
  - Why: Fullscreen_Display (no letterboxing) and parity across devices.
  - Verify: layout tests at several `Size` values; E2E screenshots.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Decide the font-size policy explicitly**: honor it by default; if the HUD can't take it, clamp it instead of disabling it app-wide. flutter/flame
  - Why: accessibility asks for large-text support; disabling it in menus and settings hurts the people who need it.
  - Verify: test with `TextScaler.linear(1.3)` and `2.0` without overflow; `rg "textScaler|withNoTextScaling" lib`.
  - Priority: important.
  - Source: https://docs.flutter.dev/ui/accessibility/ui-design-and-styling

```
MaterialApp(
  builder: (context, child) {
    final mq = MediaQuery.of(context);
    return MediaQuery(
      data: mq.copyWith(
        textScaler: mq.textScaler.clamp(minScaleFactor: 1.0, maxScaleFactor: 1.3),
      ),
      child: child!,
    );
  },
);
// For the in-game HUD (only if essential): MediaQuery.withNoTextScaling(child: hud)
```

- [ ] **Test the system "Display size" setting** (it changes density, not the font); if it breaks proportions, normalize with the stable density (`DisplayMetrics.DENSITY_DEVICE_STABLE`). agnostic
  - Why: with Display size enlarged, controls with a fixed logical size no longer line up with the world.
  - Verify: Settings › Display › Display size at minimum and maximum; proportion test.
  - Priority: nice-to-have.

- [ ] **Test on large screens, foldables and multi-window** (7.6" foldable emulator, Pixel C-class tablet, desktop window). agnostic
  - Why: the quality guidelines name these emulators as a minimum; adaptive quality also improves Play ranking.
  - Verify: AVD Manager; Play Console › Quality › large screens.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

Phase 3 Lifecycle, state and saving

- [ ] **Pause the game loop in the background**: Flame does it by default (`pauseWhenBackgrounded = true`); if you override `lifecycleStateChange` , always call `super` . flutter/flame
  - Why: avoids huge `dt` jumps on return and battery drain (Sleep_Resume, Lock_Resume).
  - Verify: lock and unlock, go to Recents and back; `rg "lifecycleStateChange|pauseWhenBackgrounded" lib` .
  - Priority: critical .
  - Source: https://docs.flame-engine.org/latest/flame/game.html

```
@override
void lifecycleStateChange(AppLifecycleState state) {
  super.lifecycleStateChange(state); // keeps Flame's automatic pause
  switch (state) {
    case AppLifecycleState.paused:
    case AppLifecycleState.hidden:
    case AppLifecycleState.detached:
      audio.pauseAll();
    case AppLifecycleState.resumed:
      if (session.isPlaying) { session.pause(); overlays.add('pause'); }
    case AppLifecycleState.inactive:
      break;
  }
}
```

- [ ] **When returning from the background, show a pause menu instead of resuming hot.** agnostic
  - Why: the player doesn't lose the run to a notification or a phone call.
  - Verify: mid-run, pull down the notification shade and go Home; the pause menu appears on return.
  - Priority: important .

- [ ] **Observe the lifecycle from the gameplay screen too** (`WidgetsBindingObserver`) for audio and timers, and adopt the current state when the screen opens. flutter/flame
  - Why: an audio hold "stuck" from a previous run leaves the game mute; `Timer.periodic` keeps running in the background.
  - Verify: widget test with `tester.binding.handleAppLifecycleStateChanged(...)` .
  - Priority: important .

```
class _GameScreenState extends State<GameScreen> with WidgetsBindingObserver {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _sync(WidgetsBinding.instance.lifecycleState); // adopt the current state
  }
  @override
  void didChangeAppLifecycleState(AppLifecycleState s) => _sync(s);
  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    _ticker?.cancel();
    super.dispose();
  }
}
```

- [ ] **Save progress and stats at safe moments** (level end, pause, exit) with fail-soft writes. agnostic
  - Why: State_Preservation; a storage exception must not crash the run.
  - Verify: kill the app from Recents mid-level and reopen it; test the persistence layer with `SharedPreferences.setMockInitialValues` .
  - Priority: important .

- [ ] **Decide what happens to the current run if the system kills the process** (restore it, or explicitly return to the menu). agnostic
  - Why: the quality guidelines ask to restore state if only a few minutes have passed.
  - Verify: `adb shell am kill <package>` with the app in the background, then return.
  - Priority: nice-to-have.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Offer "Clear data / stats" with confirmation.** agnostic
  - Why: user control; the privacy policy must explain how to delete data.
  - Verify: flow in Settings; reset test.
  - Priority: nice-to-have.

- [ ] **Configure Auto Backup** (`android:dataExtractionRules` and `android:fullBackupContent`) to include preferences. agnostic
  - Why: App_Data_Backup; progress survives a phone change.
  - Verify: `adb shell bmgr backupnow <package>`, uninstall, reinstall and check.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

Phase 4 Audio

- [ ] **Make audio fail-soft**: if init or an asset fails, the game keeps going and only that voice goes mute. agnostic
  - Why: codec or device failures must not cause a crash.
  - Verify: test with a missing asset; `try/catch` in `AudioManager`.
  - Priority: important.

- [ ] **Request and release audio focus, and react to calls or other apps.** agnostic
  - Why: Audio_Focus_Request and Audio_Focus_Change.
  - Verify: play music in another app or take a call during a run.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

```
// audioplayers
await AudioPlayer.global.setAudioContext(AudioContext(
  android: const AudioContextAndroid(
    usageType: AndroidUsageType.game,
    contentType: AndroidContentType.music,
    audioFocus: AndroidAudioFocus.gain,
  ),
));
```

- [ ] **Silence everything in the background and don't use a foreground service** (games don't play audio in the background). agnostic
  - Why: Background_Service_Optimization; nobody wants to hear the game with the screen locked.
  - Verify: go Home and lock the device; silence is immediate.
  - Priority: critical.

- [ ] **Release players in order** (`await pause()` then `dispose()`; never `stop` and `dispose` without awaiting). flutter/flame
  - Why: avoids race conditions and native crashes when leaving a run.
  - Verify: enter and leave a run 20 times in a row; logcat shows no exceptions.
  - Priority: important.

- [ ] **Add sound settings** (master, separate music and SFX, mute) and persist them. agnostic
  - Why: a basic player expectation; many people play muted.
  - Verify: test of the settings layer; restarting the app keeps the mix.
  - Priority: nice-to-have.

- [ ] **Preload SFX and loops** and pick suitable formats (short OGG or MP3; procedural audio where it fits). agnostic
  - Why: Audio_Playback_Start (under 1 s) and no stutter on the first shot.
  - Verify: profile the first sound event.
  - Priority: nice-to-have.

Phase 5 Input and predictive back

- [ ] **Enable predictive back**: `android:enableOnBackInvokedCallback="true"` on `<application>`; when targeting 36 the animation is on by default and `onBackPressed` is no longer called. agnostic ⚠ verify still current
  - Why: legacy BACK interception stops working on Android 16.
  - Verify: merged manifest; back gesture on Android 14+ (the preview of the menu or Home appears).
  - Priority: important.
  - Source: https://docs.flutter.dev/platform-integration/android/predictive-back · https://developer.android.com/guide/navigation/custom-back/predictive-back-gesture

- [ ] **Protect the current run with `PopScope`** (declarative `canPop`): a back gesture pauses, asks for confirmation and saves the attempt before leaving. flutter/flame
  - Why: State_Preservation asks to prevent accidental loss through back navigation; edge gestures collide with joysticks.
  - Verify: back gesture mid-run, with both gesture and button navigation; widget test with `tester.binding.handlePopRoute()`.
  - Priority: important.

```
PopScope(
  canPop: false,
  onPopInvokedWithResult: (didPop, _) async {
    if (didPop) return;
    game.pause();
    final leave = await showExitDialog(context);
    if (leave && context.mounted) {
      await saveAttempt();
      Navigator.of(context).pop();
    }
  },
  child: GameWidget(game: game),
)
```

- [ ] **Follow standard navigation**: back returns to the previous screen and exits from the main menu; no custom "back" buttons as the only way out. agnostic
  - Why: Back_Button_Nav and Back_Gesture_Nav.
  - Verify: walk through every screen pressing back.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Make multi-touch robust**: track pointers by id, always forward `move` and `up`, and never leave "stuck" pointers in pinches or with two joysticks. flutter/flame
  - Why: a lost `up` leaves a control pinned or a ghost pinch.
  - Verify: gesture tests with several pointers and different release orders; manual tests releasing in reverse order.
  - Priority: important.

- [ ] **Keep touch controls out of the system gesture areas** (edges), or make them configurable (position and swappable side). agnostic
  - Why: the edges trigger the back or Home gesture.
  - Verify: `MediaQuery.systemGestureInsetsOf`; control customizer.
  - Priority: nice-to-have.

- [ ] **Offer optional haptics** behind an injectable interface (so tests can use a fake). flutter/flame
  - Why: improves the feel; some players turn it off.
  - Verify: test with a fake `Haptics`; on/off setting.
  - Priority: nice-to-have.

Phase 6 Performance

- [ ] **Target 60 FPS on mid-range devices** (16.7 ms per frame) and measure it on a real device in `--profile` mode. agnostic
  - Why: Rendering_Performance.
  - Verify: `flutter run --profile` with DevTools › Performance; your own benchmark (`--dart-define=BENCH=true`) that prints p50 and p95.
  - Priority: important .
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Don't allocate memory in `update()` or `render()`** : cache `Paint` , `Path` , `TextPainter` and shaders, and reuse segments or particles with pools. flutter/flame
  - Why: GC pressure causes stutter.
  - Verify: code review; DevTools › Memory shows no GC spikes during play.
  - Priority: important .

- [ ] **Bound the cost of heavy computations** (similarity, AI, generation): run them at a bounded rate, off the frame or in an `Isolate` . agnostic
  - Why: avoids ANRs and long frames.
  - Verify: benchmark of the function; timings in the log.
  - Priority: important .

- [ ] **Show feedback if startup takes more than 2 s** and preload assets outside of gameplay. agnostic
  - Why: App_Startup_Time.
  - Verify: timed cold start; Play Console › Android vitals › Startup.
  - Priority: important .
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Watch the AAB size** (compress videos and large images; if it grows a lot, use Play Asset Delivery). agnostic
  - Why: download size affects conversion and has per-module limits on Play.
  - Verify: `ls -la build/app/outputs/bundle/release/` ; Play Console › App bundle explorer › download size.
  - Priority: nice-to-have .

- [ ] **Check temperature and battery drain** in 5–10 minute sessions. agnostic
  - Why: Power_Management; long play sessions heat up the device.
  - Verify: manual test; Android Studio › Power Profiler.
  - Priority: nice-to-have .

Phase 7 Accessibility and internationalization

- [ ] **Label controls with `Semantics` or `semanticsLabel`** (menu buttons, settings, icons). flutter/flame
  - Why: Content_Description; TalkBack reads the menus.
  - Verify: TalkBack on; `rg "Semantics|semanticsLabel" lib`; `meetsGuideline(labeledTapTargetGuideline)` in tests.
  - Priority: important.
  - Source: https://docs.flutter.dev/ui/accessibility-and-internationalization/accessibility

- [ ] **Use touch targets of 48 dp or more and contrast of 4.5:1 or more** (3:1 for large text). agnostic
  - Why: Touch_Target_Size and Visual_Contrast.
  - Verify: Accessibility Scanner; `meetsGuideline(androidTapTargetGuideline)` and `textContrastGuideline`.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

```
testWidgets('a11y', (tester) async {
  final handle = tester.ensureSemantics();
  await tester.pumpWidget(const MyApp());
  await expectLater(tester, meetsGuideline(androidTapTargetGuideline));
  await expectLater(tester, meetsGuideline(labeledTapTargetGuideline));
  await expectLater(tester, meetsGuideline(textContrastGuideline));
  handle.dispose();
});
```

- [ ] **Don't convey information through color alone** (shape, icon or supporting text); consider a color-blind mode. agnostic
  - Why: in color games or games with treasure types, about 8 % of men have a color vision deficiency.
  - Verify: color-blindness simulation in Developer options.
  - Priority: nice-to-have.

- [ ] **Externalize all strings** and choose the i18n mechanism: `flutter_localizations` with ARB (recommended) or a per-locale strings class for an MVP. flutter/flame
  - Why: translate later without touching the UI; strings scattered through widgets make the change harder.
  - Verify: `rg "Text\('" lib` returns nothing outside the strings layer; `l10n.yaml` and `lib/l10n/*.arb`.
  - Priority: important.
  - Source: https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization

- [ ] **Declare per-app language** (`localeConfig` or `supportedLocales`) and test long strings (German, Spanish). agnostic
  - Why: Android 13+ lets users choose the language per app; long strings break buttons.
  - Verify: Settings › Apps › Language; screenshots in each language.
  - Priority: nice-to-have.

- [ ] **Limit line length in long texts** (45 to 75 characters) in story or help screens. agnostic
  - Why: Line_Length.
  - Verify: visual review on a tablet.
  - Priority: nice-to-have.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

Phase 8 Errors, crashes and symbols

- [ ] **Install global error handlers** (`FlutterError.onError` and `PlatformDispatcher.instance.onError`) with local logging; no network if the policy says "offline". flutter/flame
  - Why: Dart exceptions don't bring the app down, so they go unnoticed.
  - Verify: `rg "FlutterError.onError|PlatformDispatcher.instance.onError" lib`; force an exception in debug.
  - Priority: important.

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  FlutterError.onError = (d) { FlutterError.presentError(d); localLog(d.exceptionAsString(), d.stack);
};
  PlatformDispatcher.instance.onError = (e, s) { localLog('$e', s); return true; };
  runApp(const MyApp());
}
```

- [ ] **If you obfuscate Dart, keep the symbols for every version** (`--obfuscate --split-debug-info=symbols/<version>`) outside git, and upload native debug symbols to Play. flutter/flame
  - Why: without symbols, Android vitals stack traces can't be read.
  - Verify: one symbols folder per versionCode; Play Console › App bundle › native debug symbols.
  - Priority: important.
  - Source: https://docs.flutter.dev/deployment/android

- [ ] **Decide whether to use a crash SDK** (Crashlytics, Sentry) **or only Android vitals**, and reflect it in the policy and in Data safety. agnostic
  - Why: a crash SDK collects data, and declaring it wrong in Data safety is grounds for rejection.
  - Verify: dependencies vs. privacy policy vs. Data safety.
  - Priority: important.

- [ ] **Don't log personal data**; `debugPrint` only for diagnostics; no `print` in production. agnostic
  - Why: Sensitive_Data_Logging; lint rules catch it.
  - Verify: `rg "print\(" lib`; `avoid_print` in `analysis_options.yaml`.
  - Priority: important.

- [ ] **Review the pre-launch report and Android vitals** (crashes, ANRs, slow frames) before going to production. agnostic
  - Why: Stability_ANR.
  - Verify: Play Console › Testing › Pre-launch report; Quality › Android vitals.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

Phase 9 Testing and CI

- [ ] **Keep `flutter analyze` warning-free** (with `flutter_lints` or stricter). flutter/flame
  - Why: catches cheap mistakes before the device does.
  - Verify: `flutter analyze` finishes with no issues.
  - Priority: important.

- [ ] **Write unit tests for the pure logic** (rules, scoring, seeded procedural generation, level validator) that call the shipped code without reimplementing formulas. agnostic
  - Why: protects balance and rules against regressions.
  - Verify: `flutter test`; requirement → test matrix in the spec.
  - Priority: important.

- [ ] **Prove every level is solvable** with an automated headless player or a validator. agnostic
  - Why: avoids shipping impossible levels.
  - Verify: a level-validation test that prints a report.
  - Priority: important.

- [ ] **Write widget tests for layout and flow** (several sizes, font at 1.3 and 2.0, simulated insets, navigation and back). flutter/flame
  - Why: locks in the guarantees from phases 2 and 5.
  - Verify: `tester.view.physicalSize = ...`, `MediaQueryData(textScaler: ...)` and `handlePopRoute()`.
  - Priority: important.

- [ ] **If there are shaders, add a parity test** between the Dart model and the shader (with a defined tolerance). flutter/flame
  - Why: logic must not read GPU pixels; the model is the source of truth.
  - Verify: a test that loads `FragmentProgram` headless and compares.
  - Priority: nice-to-have.

- [ ] **Generate E2E screenshots automatically** (Playwright on the release web build, or integration_test on Android). flutter/flame
  - Why: catches visual regressions and produces gameplay screenshots.
  - Verify: `npm test` in `e2e/` inside `verify.sh`.
  - Priority: nice-to-have.

- [ ] **Add CI on every PR** that runs `verify.sh` (or at least analyze and tests); site publishing goes in a separate workflow. agnostic
  - Why: a Pages-only workflow doesn't protect the code.
  - Verify: `.github/workflows/ci.yml` green on the PR; `main` protected.
  - Priority: important.

```
name: ci
on: [pull_request, push]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with: { channel: stable }
      - run: ./verify.sh --no-apk
```

- [ ] **Test on representative devices and emulators**: a physical mid-range device, the latest Android, a foldable, a tablet and a 16 KB emulator. agnostic
  - Why: this is the test environment the quality guidelines ask for.
  - Verify: matrix recorded in the acceptance script.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

Phase 10 Signing and release

- [ ] **Create the upload key** (RSA 2048 bits or more, alias `upload`) and keep it **out of git and off the shared machine**, with an encrypted backup. agnostic
  - Why: losing the key means requesting a reset; if it leaks, someone can sign in your name.
  - Verify: `git check-ignore android/key.properties *.jks`; backup checked.
  - Priority: critical.
  - Source: https://developer.android.com/studio/publish/app-signing

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA \
        -keysize 2048 -validity 10000 -alias upload
```

- [ ] **Configure release signing with `key.properties`**, tolerant of the file being missing (for CI and test builds) and **without the debug key in release**. flutter/flame
  - Why: `.toString()` on a null value produces `"null"` and breaks the build without the key; a release signed with the debug key can't be published.
  - Verify: `android/app/build.gradle.kts`; `apksigner verify --print-certs` on the APK generated from the AAB.
  - Priority: critical.
  - Source: https://docs.flutter.dev/deployment/android

```
import java.util.Properties
import java.io.FileInputStream

val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}
android {
    signingConfigs {
        create("release") {
            keyAlias = keystoreProperties.getProperty("keyAlias")
            keyPassword = keystoreProperties.getProperty("keyPassword")
            storeFile = keystoreProperties.getProperty("storeFile")?.let { file(it) }
            storePassword = keystoreProperties.getProperty("storePassword")
        }
    }
    buildTypes {
        release { signingConfig = signingConfigs.getByName("release") }
    }
}
```

```
# android/.gitignore
key.properties
**/*.keystore
**/*.jks
```

- [ ] **Enable Play App Signing** (mandatory for new apps with AAB). agnostic
  - Why: Google holds the app signing key; if the upload key is lost it can be reset. The future hybrid PQC signing (Android 17) will be offered by Play.
  - Verify: Play Console › Setup › App integrity › App signing.
  - Priority: critical.
  - Source: https://developer.android.com/studio/publish/app-signing

- [ ] **Always publish an AAB** (`flutter build appbundle --release`); APKs only for testing outside Play (`--split-per-abi`). agnostic
  - Why: AAB is mandatory on Play for new apps.
  - Verify: `build/app/outputs/bundle/release/app-release.aab` ; test it with `bundletool` or on internal testing.
  - Priority: critical .
  - Source: https://docs.flutter.dev/deployment/android

- [ ] **Check the final AAB**: target and min SDK, permissions, versionCode, 16 KB and no debug libraries. agnostic ⚠ verify still current
  - Why: Production_Build_Quality; the last check before uploading.
  - Verify: release merged manifest; `readelf -lW` on the `.so` files; Play Console › App bundle explorer.
  - Priority: critical .

- [ ] **Write per-language release notes** in the repo (`release_notes/<version>-<lang>.txt`). agnostic
  - Why: traceability and direct copy-paste into the console.
  - Verify: one file per version and language.
  - Priority: nice-to-have .

Phase 11 Google Play publishing and policies

- [ ] **Publish the privacy policy at a stable URL** (e.g. `https://<domain>/privacy.html` with GitHub Pages), bilingual and consistent with permissions, SDKs and local storage. agnostic
  - Why: Play requires it; a policy that doesn't match real behavior gets rejected.
  - Verify: URL reachable; compare permissions and dependencies against the text.
  - Priority: critical.

- [ ] **Fill in Data safety** according to what the game really does (even if it collects nothing). agnostic
  - Why: mandatory for every published app.
  - Verify: Play Console › App content › Data safety.
  - Priority: critical.
  - Source: https://support.google.com/googleplay/android-developer/answer/10787469

- [ ] **Complete the content rating questionnaire** (IARC) and redo it if the content changes. agnostic
  - Why: an app can't be published without a rating.
  - Verify: Play Console › App content › Content rating.
  - Priority: critical.
  - Source: https://support.google.com/googleplay/android-developer/answer/9859655

- [ ] **Declare target audience, ads and access**; if minors are in the audience, review Families and the minors policies. agnostic ⚠ verify still current
  - Why: these declarations are mandatory; the minors policies were tightened in 2026.
  - Verify: Play Console › App content (every section green).
  - Priority: critical.

- [ ] **Prepare the listing**: title, short and full description (per language), 512 px icon, 1024×500 feature graphic (no small text or screenshots), screenshots in the game's real orientation, and an optional video. agnostic
  - Why: Play_Feature_Graphic and Play_Misleading_Content.
  - Verify: Play Console › Main store listing; screenshots stay legible when scaled down.
  - Priority: critical.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Choose the "Game" category and its subcategory** on Play (in addition to `appCategory` in the manifest). agnostic
  - Why: discoverability, and Play can manage the game flag with an AAB.
  - Verify: Play Console › Store settings.
  - Priority: important.

- [ ] **If the game is paid or has IAP**, set up the payments profile, per-country prices and taxes; for IAP, use Play Billing. agnostic
  - Why: without a payments profile you can't set prices; IAP outside Play Billing violates policy.
  - Verify: Play Console › Setup › Payments profile; Monetize › Products.
  - Priority: critical (if applicable).

- [ ] **Go through internal testing and, if the account requires it, closed testing** before production (new personal accounts need testers for several days before production). agnostic ⚠ verify still current
  - Why: the pre-launch report and real feedback arrive before every user gets the build.
  - Verify: Play Console › Testing › Internal or closed testing.
  - Priority: important.

- [ ] **Confirm Android developer verification** (identity and package registration). agnostic ⚠ verify still current
  - Why: already required in some countries; the global rollout is planned for 2027.
  - Verify: Play Console › Settings › Developer account.
  - Priority: important.
  - Source: https://developer.android.com/developer-verification

- [ ] **Publish a minimal promo site** (page, video, Play link and policy) deployed by CI. agnostic
  - Why: it serves as the privacy URL and as a marketing destination.
  - Verify: Pages workflow green; your own domain with HTTPS.
  - Priority: nice-to-have.

Phase 12 Post-launch maintenance

- [ ] **Review the target API requirement every year** (Play usually raises it in August, with an extension until November) and update Flutter, AGP and NDK. agnostic ⚠ verify still current
  - Why: below the required level you can't publish updates, and existing apps stop being visible to new users.
  - Verify: https://support.google.com/googleplay/android-developer/answer/11926878; Play Console › Dashboard (policy notices).
  - Priority: critical.

- [ ] **Read the behavior changes of every new Android release** (edge-to-edge, orientation, back, 16 KB...) and test with the compatibility toggles. agnostic
  - Why: Android_Platform_Compatibility.
  - Verify: emulator on the latest API; Developer options › App Compatibility Changes.
  - Priority: important.
  - Source: https://developer.android.com/about/versions/16/behavior-changes-16

- [ ] **Keep dependencies up to date** (`flutter pub outdated`), prioritizing those that ship native code. flutter/flame
  - Why: SDK_Maintenance; native plugins are the usual source of 16 KB problems and crashes.
  - Verify: `flutter pub outdated`; each native plugin's changelog.
  - Priority: important.
  - Source: https://developer.android.com/docs/quality-guidelines/core-app-quality

- [ ] **Watch Android vitals and reviews** and respond to bugs that reproduce. agnostic
  - Why: Play_User_Reviews and Stability_ANR; vitals thresholds affect visibility.
  - Verify: Play Console › Quality › Android vitals, and Ratings and reviews.
  - Priority: important.

- [ ] **Keep the docs in sync with the published version** (roadmap checked off, README without contradictory versions, state up to date). agnostic
  - Why: stale docs mislead agents, and the author too, months later.
  - Verify: the version in the README, the plan and `pubspec.yaml` match; the release-phase roadmap is closed.
  - Priority: nice-to-have.

- [ ] **Feed each game's lessons back into this template or skill** (what was solved, what was missing). agnostic
  - Why: continuous improvement across projects.
  - Verify: an entry in the skill's changelog.
  - Priority: nice-to-have.

Notes for turning this into a skill

Suggested name: `android-game-creation`

When to use it (trigger)

- The user wants to create, prepare or audit a mobile game for Android ("new game", "publish on Play", "release checklist", "review my game before uploading").
- There is a Flutter repo with `flame` in `pubspec.yaml`, or an Android project whose main purpose is a game.
- Don't use it for non-game apps: `appCategory="game"` and its orientation exceptions don't apply there.

Inputs the skill should ask for

1. Mode: **new project** (phases 0 to 12) or **audit** of an existing repo (phases 1 to 11 against the code).
2. Game name and `applicationId` (reverse domain), and whether it's already published on Play.
3. Orientation (portrait or landscape) and whether it must support tablets or foldables.
4. Stack: Flutter + Flame (default) or another (only the `agnostic` items apply).
5. Monetization: free, one-time purchase, IAP or ads (changes Data safety, Billing and Families).
6. Data and network: offline or online, analytics, crash SDK (changes the policy and Data safety).
7. UI and listing languages.
8. Target audience (does it include minors?).
9. The agent's autonomy level and stop rules (what the owner always decides).

Steps that can be automated (a `verify.sh` or `audit.sh` script)

Check	How
Analyze and tests	`flutter analyze` · `flutter test`
Effective SDKs	`flutter build appbundle` and read `<uses-sdk>` from the release merged manifest
16 KB	unzip the AAB and `readelf -lW base/lib/*/*.so | awk '/LOAD/{print $NF; exit}'` ≥ `0x4000`
Manifest flags	`grep` for `appCategory="game"`, `enableOnBackInvokedCallback="true"`, `exported`, `uses-permission`
Edge-to-edge	`rg "immersiveSticky|windowOptOutEdgeToEdge"` (warn if found with target ≥ 36)
Default icon	`md5sum mipmap-*/ic_launcher.png` vs. the Flutter template
Light splash	`grep "Theme.Light" res/values/styles.xml` in a dark game (warning)
Signing	`signingConfigs.getByName("debug")` in release (error); `key.properties` and `*.jks` in `.gitignore`
Version	`version:` in `pubspec.yaml` vs. README and plan; versionCode greater than the last one uploaded (if known)
Lifecycle	`rg "lifecycleStateChange|WidgetsBindingObserver|pauseWhenBackgrounded"`; if overridden, check for `super.`
Back	`rg "PopScope" lib/**/game*`
A11y and i18n	`rg "Semantics"`; `rg "Text\('"` outside the strings layer; `meetsGuideline` tests
Errors	`rg "FlutterError.onError|PlatformDispatcher.instance.onError"`; `rg "print\("`
Permissions and policy	list of `uses-permission` and network-using dependencies vs. the text of `privacy.html`
CI	a `.github/workflows/*` exists that runs analyze and test (not just Pages)

Steps that stay with the human or Play Console (the skill only reminds about them): Play App Signing, Data safety, IARC rating, target audience, listing and graphics, payments profile, internal or closed testing, developer verification, game feel and difficulty (acceptance script), testing on a physical device.

Maintaining the skill: every item marked ⚠ verify still current must be checked against its source whenever the skill runs. Record in the skill the date of the last review (this version: Oct 19, 2026).